THE VIBE CODER'S .md FILE STARTER GUIDE

A beginner-friendly guide to SECURITY.md, CODE_STYLE.md, DATABASE.md and API.md

Why these files matter

AI coding tools can generate code quickly, but they still need project-specific rules. These Markdown files give your coding agent persistent context about security, code conventions, data, and APIs so it does not have to guess every time you start a new prompt.

Use this guide as a starting template, not a rulebook. Replace the examples with the actual stack, requirements, and constraints of the product you are building. Delete sections that do not apply and add project-specific rules when needed.

SECURITY.md • CODE_STYLE.md • DATABASE.md • API.md

How to use these files

Create a /docs folder in your project and place the four files inside it. Your AGENTS.md can then tell the coding agent to read the relevant documentation before making changes.

```
my-app/
├── app/
├── components/
├── lib/
└── docs/
    ├── PRD.md
    ├── AGENTS.md
    ├── DESIGN_SYSTEM.md
    ├── ARCHITECTURE.md
    ├── SECURITY.md
    ├── CODE_STYLE.md
    ├── DATABASE.md
    └── API.md
```

A simple workflow

1. Start with the sample for each file in this guide.
2. Replace placeholders with your real stack, providers, routes and project rules.
3. Tell your AI coding agent to read the relevant .md file before implementing a feature.
4. Update the documentation whenever the implementation changes.
5. Treat the files as shared context — they should describe the project you actually have, not the project you used to have.

Beginner tip

Do not add rules just because they sound technical. If your app uses Supabase, document Supabase. If it uses Firebase, document Firebase. Accurate context is more useful to an AI agent than a long generic file.

SECURITY.md

Defines how the project protects users, data, credentials and sensitive operations.

What should go inside?

- Authentication and authorization rules
- How secrets and environment variables are handled
- Input validation and server-side verification
- API and backend security rules
- Database access and data protection
- Error handling and logging rules
- Dependency and infrastructure practices
- What the AI agent must never do

Beginner-friendly sample

```
# SECURITY.md

## 1. Purpose
This file defines the security rules for this project.
All developers and AI coding agents must follow these rules.

## 2. Authentication & Authorization
- Use [Clerk / Supabase Auth / NextAuth] for authentication.
- Protected routes must verify the user's session on the server.
- Never trust a user ID or role sent only from the client.
- Check permissions before reading or modifying protected data.
- Use role-based access where the product requires different permissions.

## 3. Secrets & Environment Variables
- Store API keys and credentials in environment variables.
- Never hardcode secrets in source code.
- Never commit .env files.
- Keep a .env.example containing variable names only.
- Use separate credentials for development and production.

Example:
DATABASE_URL=
STRIPE_SECRET_KEY=
NEXT_PUBLIC_APP_URL=

## 4. Input Validation
- Validate all user-controlled input.
- Validate again on the server even if the UI already validates it.
- Use a schema validator such as Zod when appropriate.
- Reject unexpected fields and invalid data types.

## 5. API Security
- Require authentication on protected endpoints.
- Apply authorization checks before sensitive actions.
- Add rate limiting to endpoints vulnerable to abuse.
- Use HTTPS in production.
- Do not expose internal stack traces to users.
```

```
## 6. Data Protection
- Store only data the product actually needs.
- Do not log passwords, tokens or sensitive personal data.
- Use parameterized queries or the project's ORM.
- Restrict database permissions where possible.
- Use provider-supported encryption and backups.

## 7. Error Handling
User-facing errors should be useful but must not reveal:
- database credentials
- API keys
- stack traces
- internal file paths
- sensitive account information

## 8. AI Agent Rules
The coding agent must:
- Never invent or hardcode production credentials.
- Never disable authentication to make a feature work.
- Never bypass authorization checks.
- Never expose secrets in client-side code.
- Ask for clarification when a requested change conflicts with this file.
```

Customize it

Replace the authentication provider, validation library, payment provider and security requirements with the tools your project actually uses. A healthcare or finance product may also need much stricter privacy and compliance requirements.

CODE_STYLE.md

Sets the conventions the project follows so AI-generated code stays readable, predictable and consistent.

What should go inside?

- Language and framework conventions
- File, component, function and variable naming
- Formatting rules
- Component structure
- Import conventions
- Rules for comments and documentation
- Patterns to reuse or avoid

Beginner-friendly sample

```
# CODE_STYLE.md

## 1. Purpose
This file defines the coding conventions for this project.
New code should follow existing patterns unless this file says otherwise.

## 2. Stack
- TypeScript
- React
- Next.js
- Tailwind CSS
- ESLint
- Prettier

## 3. General Principles
- Prefer readable code over clever code.
- Keep functions and components focused.
- Reuse existing components before creating new ones.
- Avoid unnecessary abstractions.
- Do not duplicate logic that already has a shared utility.

## 4. Naming
Components: PascalCase
Example: UserCard.tsx

Functions and variables: camelCase
Example: getUserData, currentUser

Constants: UPPER_SNAKE_CASE
Example: MAX_RETRY_COUNT

Boolean values should read clearly:
isLoading
hasAccess
canEdit

## 5. Components
- Use functional React components.
```

```
- Keep UI, business logic and data fetching separated when practical.
- Move reusable complex logic into custom hooks or utilities.
- Define clear TypeScript types for props.
- Handle loading, error and empty states.

## 6. Formatting
- Use the project's Prettier configuration.
- Use the project's ESLint configuration.
- Do not manually reformat unrelated files.
- Keep imports organized.
- Remove unused variables and imports.

## 7. Comments
Write comments to explain WHY, not obvious WHAT.

Good:
 // Retry once because the provider occasionally returns a transient 503.

Avoid:
 // Increment count
 count++;

## 8. Before Finishing
- Run linting.
- Run type checking.
- Run relevant tests.
- Check responsive layouts when UI was changed.
- Remove debugging logs and dead code.
```

Customize it

Your code-style file should match the repository. If the existing project uses CSS Modules instead of Tailwind, or JavaScript instead of TypeScript, change the guide rather than forcing a new convention.

DATABASE.md

Explains how the application's data is structured, related, accessed and changed safely.

What should go inside?

- Database provider and ORM
- Environment and connection rules
- Core tables/models
- Relationships between data
- Naming conventions
- Indexes and constraints
- Migration and seed workflow
- Rules for production data

Beginner-friendly sample

```
# DATABASE.md

## 1. Purpose
This file describes how data is modeled and managed in this project.

## 2. Database Stack
Primary database: PostgreSQL
Provider: [Supabase / Neon / other]
ORM: Prisma
Migrations: Prisma Migrate

## 3. Environment
DATABASE_URL must come from environment variables.
Never hardcode a production connection string.

Development, staging and production should use separate databases.

## 4. Core Models

### User
Represents an application user.

Fields:
- id: unique identifier
- email: unique email address
- name: optional display name
- createdAt
- updatedAt

### Project
Represents a project created by a user.

Fields:
- id
- name
- ownerId -> User.id
- createdAt
```

```
- updatedAt

Relationship:
One User can have many Projects.
Each Project belongs to one User.

## 5. Schema Rules
- Every primary record should have a stable ID.
- Add created/updated timestamps where useful.
- Use foreign keys for relationships.
- Add unique constraints for values that must be unique.
- Add indexes to frequently queried fields when justified.
- Avoid storing the same fact in multiple places without a reason.

## 6. Migrations
When the schema changes:
1. Update the schema definition.
2. Create a migration.
3. Review the generated migration.
4. Test it locally/staging.
5. Apply it through the project's deployment process.

Never manually change the production database just to skip a migration.

## 7. Seed Data
Seed scripts may create development/test data.
Do not include real user data or production secrets in seeds.

## 8. Production Safety
- Back up important data.
- Avoid destructive schema changes without a migration plan.
- Never run reset commands against production.
- Use transactions for operations that must succeed or fail together.
- Follow SECURITY.md for access and sensitive data rules.
```

Customize it

The most useful part of DATABASE.md is your real data model. Add the actual models your product uses and explain their relationships in plain English. This gives an AI agent far more context than simply saying 'we use PostgreSQL.'

API.md

Documents how the frontend, backend and external services communicate.

What should go inside?

- Base URL and versioning
- Authentication requirements
- Endpoint naming conventions
- Request parameters and bodies
- Response shapes
- Status and error codes
- Rate limits
- Third-party integrations

Beginner-friendly sample

```
# API.md

## 1. Purpose
This file documents the API conventions and important endpoints used by this project.

## 2. Base Configuration
Development: http://localhost:3000/api
Production: https://your-domain.com/api
Response format: JSON

If the API is versioned:
Base path: /api/v1

## 3. Authentication
Protected endpoints require an authenticated session/token.

Example header:
Authorization: Bearer <token>

Never expose server-only API credentials to the browser.

## 4. Endpoint Conventions
Use nouns for resources where practical.

GET    /api/v1/projects
GET    /api/v1/projects/:id
POST   /api/v1/projects
PATCH  /api/v1/projects/:id
DELETE /api/v1/projects/:id

## 5. Example Endpoint

### POST /api/v1/projects
Creates a new project.

Authentication:
Required
```

```
Request:
{
  "name": "Creator Dashboard"
}

Success — 201:
{
  "success": true,
  "data": {
    "id": "project_123",
    "name": "Creator Dashboard"
  }
}

Validation error — 400:
{
  "success": false,
  "error": {
    "code": "INVALID_INPUT",
    "message": "Project name is required."
  }
}

## 6. Status Codes
200 - Successful request
201 - Resource created
400 - Invalid request
401 - Authentication required
403 - Authenticated but not allowed
404 - Resource not found
409 - Conflict
429 - Too many requests
500 - Unexpected server error

## 7. Error Rules
- Return a consistent error shape.
- Do not return internal stack traces.
- Give users actionable messages where safe.
- Log enough server-side context to debug failures.
- Do not log secrets or sensitive request data.

## 8. Rate Limiting
Apply rate limits to endpoints that can be abused, especially:
- authentication
- password reset
- AI generation
- public forms
- expensive searches

## 9. Third-Party APIs
For every important integration, document:
- provider name
- purpose
- required environment variables
- server/client usage
- webhook endpoints
- expected failure behavior
```

```
Example:
Stripe
Purpose: payments
Secret: STRIPE_SECRET_KEY (server only)
Webhook: /api/webhooks/stripe
```

Customize it

Do not copy example endpoints into a project that does not have them. Replace them with your actual routes. If your app uses server actions, GraphQL or tRPC instead of REST endpoints, document that pattern instead.

How to give these files to your AI coding agent

Having the files in your repository is useful, but explicitly referencing them in important prompts makes the expected behavior clearer.

Before building a feature

```
Before implementing this feature, read:
- docs/SECURITY.md
- docs/CODE_STYLE.md
- docs/DATABASE.md
- docs/API.md

Then inspect the existing codebase for relevant patterns.

Build the feature using the existing project conventions. Do not invent
new database models, API patterns, dependencies or security rules unless
they are required. If the documentation conflicts with the existing
implementation, flag the conflict before making a major change.
```

When changing the database

```
Read DATABASE.md and SECURITY.md first.

I need to add [FEATURE].

Before coding:
1. Identify which models/tables need to change.
2. Explain the proposed relationship and constraints.
3. Identify whether a migration is required.
4. Check whether the change affects authorization or sensitive data.
5. Then implement it using the existing database conventions.
```

When adding an API endpoint

```
Read API.md and SECURITY.md first.

Create an endpoint for [FEATURE].
Follow the existing response/error format.
Validate all inputs.
Apply authentication and authorization where required.
Do not expose internal errors or secrets.
Update API.md if this introduces a new public project convention.
```

Common beginner mistakes

- Copying a template without replacing the sample stack or providers.
- Letting documentation say one thing while the codebase does another.
- Putting actual API keys, passwords or connection strings inside Markdown files.
- Using SECURITY.md as a substitute for implementing security in the code.
- Documenting every tiny function instead of documenting stable project conventions.
- Making the files so long that important rules are difficult to find.
- Changing architecture or database conventions without updating the relevant documentation.

- Assuming the AI has read every file automatically; reference important files in AGENTS.md or your prompt.

The rule to remember

These files are context, not magic. They help an AI coding agent make better project-specific decisions, but you still need to review generated code, test the application, and verify security-sensitive changes.

Quick project checklist

Before you call these files finished, make sure you can answer yes to the following:

File	It should answer...	Ready?
SECURITY.md	How should this project protect users, data and secrets?	☐
CODE_STYLE.md	What should code in this repository look like?	☐
DATABASE.md	How is our data modeled and changed safely?	☐
API.md	How do parts of the product communicate?	☐

Final note

Start small. A short, accurate document is more useful than a huge template full of rules your project does not use. As your product grows, let the documentation grow with it.